



United International University

Department of CSE

Course Title: Microprocessors and Microcontrollers Laboratory

Course Code: CSE 4326

Group No: 03

Name: Md Sadique Hossain (on behalf of my team)

ID – 0112230558

Section – 'I'

Lab Report – 2

Experiment Name – Wifi Communication and building IoT-based systems Using **XAMPP** and **ESP32**

Date of Performance – 10/08/2025

Date of Submission – 17/08/2025

To - Md. Shafqat Talukder Rakin

Lecturer Dept. of Computer Science & Engineering

United International University

(a) Objective:

The objective of this experiment is to establish WiFi communication using the ESP32 microcontroller and develop an IoT-based system utilizing XAMPP for local server management. The experiment involves:

- **Sensor Data Acquisition:** Collect real-time temperature and humidity data using the DHT11 sensor.
- **Data Transmission & Storage:** Send the collected data to a local XAMPP server and store it in a MySQL database using PHP.
- **Web Interface Development:** Display the sensor data on a web page for real time monitoring.
- **IoT Cloud Integration:** Connect ESP32 to the Arduino IoT Cloud for remote monitoring of gas sensor data.
- **Remote Device Control:** Implement a dashboard for remotely controlling an LED via the IoT platform.

This experiment provides hands-on experience in IoT system development, database management, and cloud-based device control.

(b) Components Used:

Hardware Components:

- DHT11 Sensor
- OLED Display
- ESP32
- LED
- Push Button
- Resistor
- Jumper Wires

Software Used:

- Arduino IDE (for coding)
- VS Code (for server coding)
- Browser (Google Chrome)
- XAMPP

c) Theory:

This experiment focuses on WiFi communication using ESP32 to develop an IoT-based system for real-time sensor data acquisition, storage, and remote control.

- **IoT & ESP32:** IoT enables remote monitoring and automation, with ESP32 providing WiFi connectivity for sensor-based applications.
- **Sensor Data Collection:** Using DHT11 (temperature & humidity) sensors to gather real-time environmental data.
- **XAMPP & Local Server:** XAMPP (Apache, MySQL, PHP) stores sensor data in a database and serves it on a web page.
- **Web-Based Monitoring:** Updating data in Google Spreadsheet, where anyone can monitor the data from anywhere in the world.
- **Data Visualization:** Showing data on OLED display collected from sensors.

(d) Problem/Design Solve Procedure:

At the start of the experiment, we installed the required libraries for the Arduino IDE, including those for the OLED display and the DHT11 sensor. Once the environment was properly set up, we carried out the experiment step by step.

1. Connect a push button and an LED to ESP32 pins. Write a program where the LED turns on when the button is pressed and turns off when it's released:

- Connected the LED light with resistor to the ESP32.
- Connected the Push Button to the ESP32.
- Wrote necessary program to turn on the light when button is pressed.

```
1 void setup() {
2   pinMode(2, OUTPUT);
3   pinMode(4, INPUT_PULLUP);
4 }
5
6 void loop() {
7   if (digitalRead(4) == LOW) {
8     digitalWrite(2, HIGH);
9   } else {
10    digitalWrite(2, LOW);
11  }
12 }
13
```

Fig: Arduino Code for an LED using a Push Button

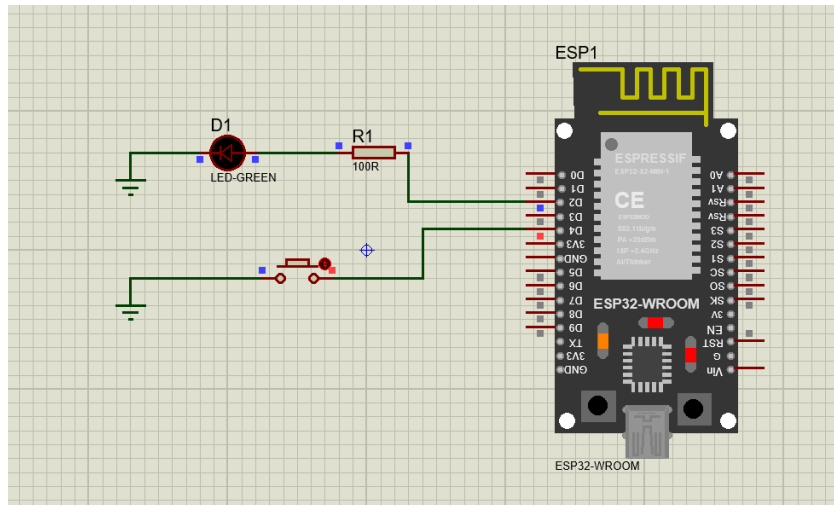


Fig: Button Not Pressed

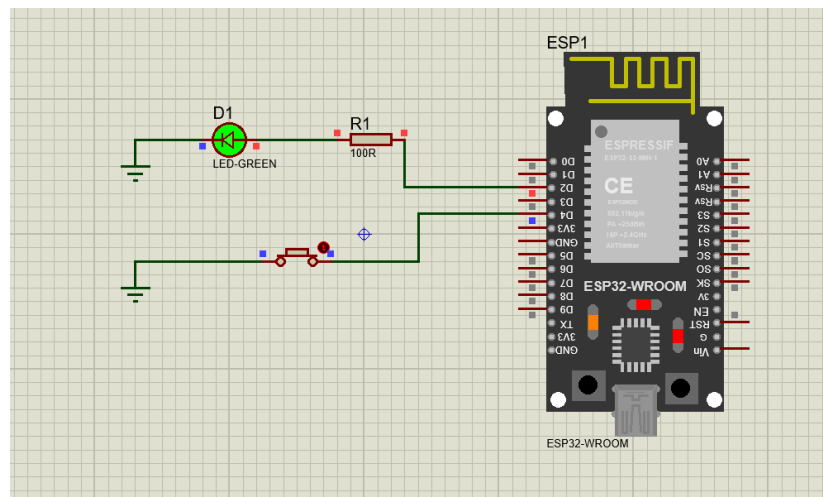


Fig: Button Pressed

2. Connect a DHT11 or DHT22 sensor to a ESP32 pin and read temperature and humidity data. Display this data on the console:

- Connecting DHT11 Sensor to the ESP32.
- Write necessary program to get temperature and humidity data from sensor.
- Printing the temperature and humidity data on the console.
- Storing the temperature and humidity data on the excel sheet based on reference video. But we ignore the usage of two switch and consider that these two switch will be always off in postData variable.

```

01_test_DHT11_and_Switches.ino
1  #include <WiFi.h>
2  #include <HTTPClient.h>
3  #include <DHT.h>
4  #define DHTPIN 19
5  #define DHTTYPE DHT11
6  DHT dht11(DHTPIN, DHTTYPE);
7  String URL =
8    "https://script.google.com/macros/s/AKfycbzsr0F6b8CaJbq4aAFT0SuoTTqe-_VQIzLT3Qpcge3yGtHG--3w1VuI9x8qIKFg1UU/exec";
9  const char* ssid = "UIU-STUDENT";
10 const char* password = "12345678";
11 int temperature = 0;
12 int humidity = 0;
13
14 void setup() {
15   Serial.begin(115200);
16   dht11.begin();
17   connectWiFi();
18 }
19

```

```

19
20 void loop() {
21   if (WiFi.status() != WL_CONNECTED) {
22     connectWiFi();
23   }
24   delay(2000);
25   Load_DHT11_Data();
26   delay(2000);
27   String postData = "?sts=write&srs=Success&temp=" + String(temperature)
28     + "&humd=" + String(humidity) + "&swtc1=Off&swtc2=Off";
29   HTTPClient http;
30   http.begin(URL);
31   http.addHeader("Content-Type", "application/x-www-form-urlencoded");
32   int httpCode = http.POST(postData);
33   String payload = http.getString();
34   Serial.print("URL : ");
35   Serial.println(URL);
36   Serial.print("Data: ");
37   Serial.println(postData);
38   Serial.print("httpCode: ");
39   Serial.println(httpCode);
40   Serial.print("payload : ");
41   Serial.println(payload);
42   Serial.println("-----");
43   delay(10000);
44 }
45

```

```

45
46 void Load_DHT11_Data() {
47   temperature = dht11.readTemperature();
48   humidity = dht11.readHumidity();
49   if (isnan(temperature) || isnan(humidity)) {
50     Serial.println("Failed to read from DHT sensor!");
51     temperature = 0;
52     humidity = 0;
53   }
54   Serial.printf("Temperature: %d °C\n", temperature);
55   delay(2000);
56   Serial.printf("Humidity: %d %%\n", humidity);
57   delay(2000);
58 }
59 void connectWiFi() {
60   WiFi.mode(WIFI_OFF);
61   delay(1000);
62   WiFi.mode(WIFI_STA);
63   WiFi.begin(ssid, password);
64   Serial.println("Connecting to WiFi");
65   while (WiFi.status() != WL_CONNECTED) {
66     delay(500);
67     Serial.print(".");
68   }
69   Serial.print("connected to : ");
70   Serial.println(ssid);
71   Serial.print("IP address: ");
72   Serial.println(WiFi.localIP());
73 }

```

Fig 2.1: Arduino Code for DTH11 Sensor

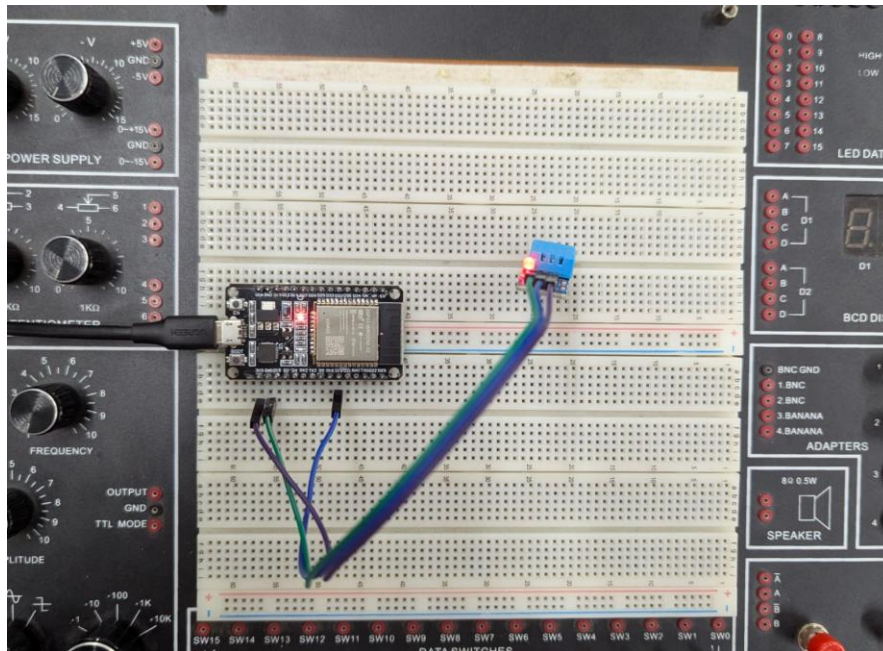


Fig 2.2: Connected the DHT11 Sensor to the ESP32

e.g. ESP32_DHT11_Data ☆ 📁 Saved to Drive

File Edit View Insert Format Data Tools Extensions Help

100% 123 Default...

	A	B	C	D	E	F	G	H
1	Timestamp	Status	Source	Temperature	Humidity	Switch1	Switch2	
2	8/17/2025 12:40:14	N/A	Success	26	56	Off	Off	
3	8/17/2025 12:40:35	N/A	Success	26	56	Off	Off	
4	8/17/2025 12:40:57	N/A	Success	26	56	Off	Off	
5	8/17/2025 12:41:18	N/A	Success	26	56	Off	Off	
6	8/17/2025 12:41:39	N/A	Success	33	95	Off	Off	
7	8/17/2025 12:42:00	N/A	Success	33	95	Off	Off	
8	8/17/2025 12:42:21	N/A	Success	30	95	Off	Off	
9	8/17/2025 12:42:42	N/A	Success	28	95	Off	Off	
10	8/17/2025 12:43:04	N/A	Success	27	75	Off	Off	
11	8/17/2025 12:43:26	N/A	Success	26	62	Off	Off	
12	8/17/2025 13:02:43	N/A	Success	26	60	Off	Off	
13	8/17/2025 13:03:07	N/A	Success	26	60	Off	Off	
14	8/17/2025 13:03:29	N/A	Success	26	60	Off	Off	
15	8/17/2025 13:03:53	N/A	Success	26	61	Off	Off	
16	8/17/2025 13:04:19	N/A	Success	26	61	Off	Off	
17	8/17/2025 13:04:40	N/A	Success	26	61	Off	Off	
18	8/17/2025 13:05:02	N/A	Success	26	61	Off	Off	
19	8/17/2025 13:05:24	N/A	Success	26	62	Off	Off	
20								
21								
22								
23								

Fig 2.3: Stored data in Google Spreadsheet

3. Display DHT11 or DHT22 sensor data on an I2C OLED display:

- Connect the DHT11 Sensor to the ESP32.
- Connect the OLED Display to the ESP32.
- Write necessary program to get temperature and humidity data from sensor.
- Printing the temperature and humidity data on the OLED Display.

01_Test_DHT11_and_Switches.ino

```
1  #include <DHT.h>
2  #include <Wire.h>
3  #include <Adafruit_GFX.h>
4  #include <Adafruit_SSD1306.h>
5
6  #define OLED_RESET -1
7  #define SCREEN_ADDRESS 0x3C
8  #define DHTPIN 19
9  #define DHTTYPE DHT11
10
11  DHT dht11(DHTPIN, DHTTYPE);
12  Adafruit_SSD1306 display(128, 64, &Wire, OLED_RESET);
13
14  int temperature = 0;
15  int humidity = 0;
```

```
17  void setup() {
18      Serial.begin(9600);
19
20      // Initialize I2C with custom pins for ESP32
21      Wire.begin(23, 22); // SDA = 23, SCL = 22
22
23      if (!display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
24          Serial.println(F("SSD1306 allocation failed"));
25          for (;;);
26      }
27
28      display.clearDisplay();
29      display.setTextColor(WHITE);
30      display.setTextSize(1);
31      display.display();
32
33      dht11.begin();
34      delay(1000);
35  }
36
```

```

35 }
36
37 void loop() {
38     Load_DHT11_Data();
39
40     Serial.print("Temperature: ");
41     Serial.print(temperature);
42     Serial.println(" °C");
43
44     Serial.print("Humidity: ");
45     Serial.print(humidity);
46     Serial.println(" %");
47
48     // Display on OLED
49     display.clearDisplay();
50     display.setCursor(0, 0);
51     display.setTextSize(1);
52     display.println("DHT11 Sensor Readings:");
53
54     display.setTextSize(2);
55     display.setCursor(0, 20);
56     display.print("Temp: ");
57     display.print(temperature);
58     display.println(" °C");
59
60     display.setCursor(0, 45);
61     display.print("Hum: ");
62     display.print(humidity);
63     display.println(" %");
64     display.display();
65     delay(2000);
66 }
67

```

```

68 void Load_DHT11_Data() {
69     temperature = dht11.readTemperature();
70     humidity = dht11.readHumidity();
71
72     if (isnan(temperature) || isnan(humidity)) {
73         Serial.println("Failed to read from DHT sensor!");
74         temperature = 0;
75         humidity = 0;
76     }
77 }
78

```

Fig 3.1: Arduino Code for DHT11 Sensor to showing data on a display.

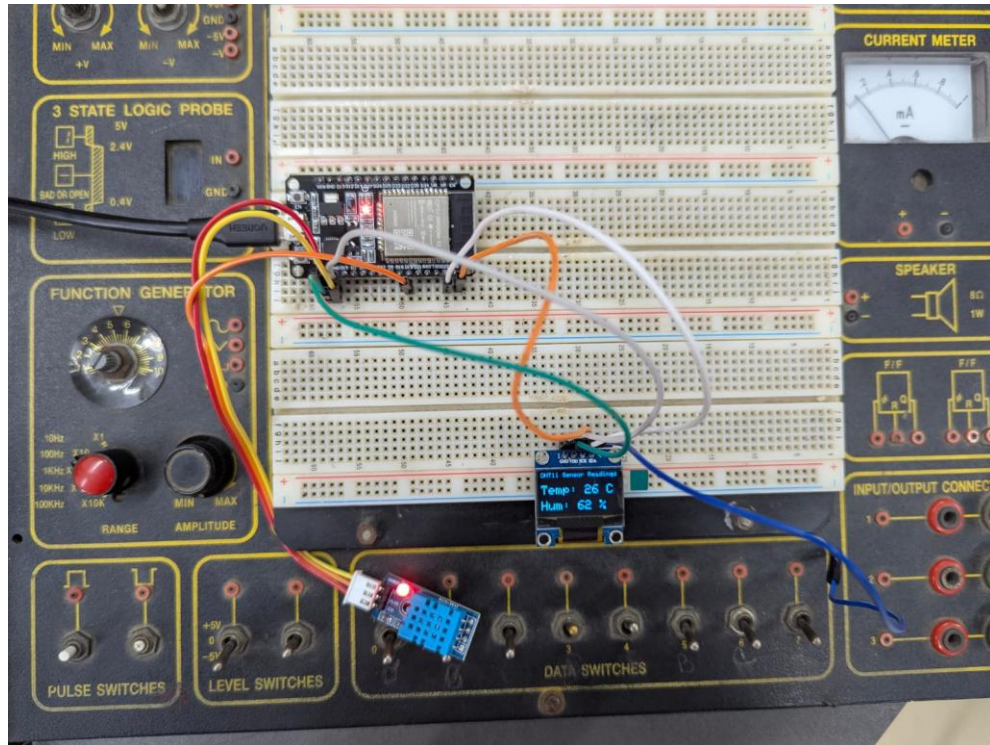


Fig 3.2: Showing the temperature and humidity data of DHT11 Sensor in OLED Display

e) Discussion:

This experiment allowed us to understand how to implement WiFi communication on the ESP32 and develop an IoT-based system with electronic devices.

- **Challenges Faced:**

Interfacing multiple components, such as the OLED display and DHT11 sensor, required careful pin configuration. Additionally, successfully uploading sensor data to a Google Spreadsheet proved to be a time-consuming process.

- **Real-Life Applications:**

- A wireless weather tracker can be created for any specific location, allowing real-time monitoring from anywhere in the world.
- The system can be integrated into smart home automation to control fans or air conditioners based on temperature and humidity readings.

Overall, this experiment significantly enhanced our understanding of building IoT-based systems using the ESP32 with WiFi connectivity.